

NUMN33: Project Part 1

Arpita Bhattacharya and Carlotta Boi -Homework Group 6

February 7, 2025

In this project, Ω is a bounded domain in $\mathbb{R}^d$ with a smooth boundary $\partial\Omega$.

Theoretical Exercises

Task 1

Give the definitions of the Sobolev spaces $H^1(\Omega)$ and $H_0^1(\Omega)$.

In order to write the definition of Sobolev spaces, we need first to give some preliminary definitions.

Definition 1. Let $\alpha \in \mathbb{N}_0^d$ and $|\alpha| = \sum_{i=1}^d \alpha_i$. Then $D_u^\alpha = \frac{\partial^{|\alpha|}}{\partial x_1^{\alpha_1} \dots \partial x_d^{\alpha_d}} u$ for $\alpha \neq 0$ and $D^0 u = u$

Definition 2. Let $u \in L^2(\Omega)$ be fixed. If there exists a $v \in L^2(\Omega)$ such that:

$$(v, \varphi)_{L^2(\Omega)} = (-1)^{|\alpha|} (u, D_\varphi^\alpha)_{L^2(\Omega)}, \forall \varphi \in C_0^\infty(\Omega),$$

then v is called the weak (partial) derivative of u . We say that $v = D^\alpha u$.

We can now give the definition of Sobolev spaces:

Definition 3. Let $k \geq 1$. $H^k(\Omega) := \{u \in L^2(\Omega) : D^\alpha u \in L^2(\Omega) \text{ for all } |\alpha| \leq k\}$.

We equip this vector space with the inner product:

$$(u, v)_{H^k(\Omega)} := \sum_{|\alpha| \leq k} (D^\alpha u, D^\alpha v)_{L^2(\Omega)}.$$

Therefore, if $k = 1$ we have: $H^1(\Omega) = \{u \in L^2(\Omega) : D^\alpha u \in L^2(\Omega) \text{ for all } |\alpha| \leq 1\}$. Note that $H^0(\Omega) := L^2(\Omega)$.

Definition 4. $H_0^k(\Omega) := \text{closure of } C_0^\infty(\Omega) \text{ in } H^k(\Omega).$

If $k = 1$, H_0^1 generalizes the concept of functions fulfilling (homogeneous) Dirichlet boundary conditions, i.e., $u = 0$ on $\partial\Omega$.

Task 2

Let $u \in C^1([a, b])$. Check that the classical derivative u' is also a weak derivative.

In Task 1 we gave the definition of weak derivative. $L^2(\Omega)$ is an Hilbert space equipped with the inner product:

$$(u, v)_{L^2(\Omega)} = \int_{\Omega} u \cdot v dx.$$

We need to prove the following:

$$\int_a^b u'(x) \cdot \varphi(x) dx = - \int_a^b u(x) \cdot \varphi'(x) dx.$$

Since $u \in C^1([a, b])$, we can integrate by parts and obtain the following:

$$\int_a^b u'(x) \cdot \varphi(x) dx = [u(x) \cdot \varphi(x)]_a^b - \int_a^b u(x) \cdot \varphi'(x) dx$$

But $[u(x) \cdot \varphi(x)]_a^b = 0$, since φ has compact support in (a, b) and therefore $\varphi(a) = \varphi(b) = 0$. Consequently we obtain that the classical derivative is a weak derivative.

Task 3

Prove that the weak derivative is unique on $L^2(\Omega)$.

Suppose $v, w \in L^2(\Omega)$ are both weak partial derivatives of u , then:

$$(-1)^{|\alpha|} \int_{\Omega} u \cdot D^\alpha \varphi dx = \int_{\Omega} v \cdot \varphi dx = \int_{\Omega} w \cdot \varphi dx, \forall \varphi \in C_0^\infty(\Omega).$$

This implies that:

$$\int_{\Omega} (v - w) \varphi dx = 0, \forall \varphi \in C_0^\infty(\Omega).$$

Therefore, because $C_0^\infty(\Omega)$ is dense in $L^2(\Omega)$, we have:

$$v - w = 0 \rightarrow v = w \text{ in } L^2(\Omega).$$

This can also be written as:

$$\begin{aligned} (-1)^{|\alpha|}(u, D^\alpha \varphi)_{L^2(\Omega)} &= (v, \varphi)_{L^2(\Omega)} = (w, \varphi)_{L^2(\Omega)}, \\ \text{then } (v - w, \varphi)_{L^2(\Omega)} &= 0. \end{aligned}$$

The difference of two weak derivatives is orthogonal to all $\varphi \in C_0^\infty(\Omega)$, therefore it must be zero in $L^2(\Omega)$.

Task 4

Prove that $u \mapsto \|\nabla u\|_{L^2(\Omega)}$ and $u \mapsto \|u\|_{H^1(\Omega)}$ are equivalent norms on $H_0^1(\Omega)$.

$H^1(\Omega)$ is equipped with the inner product:

$$(u, v)_{H^1(\Omega)} = (u, v)_{L^2(\Omega)} + \sum_{i=1}^d \left(\frac{\partial}{\partial x_i} u, \frac{\partial}{\partial x_i} v \right)_{L^2(\Omega)} =: (u, v)_{L^2(\Omega)} + (\nabla u, \nabla v)_{L^2(\Omega)}.$$

Therefore we can write the norm as follows:

$$(u, u)_{H^1(\Omega)} = (u, u)_{L^2(\Omega)} + (\nabla u, \nabla u)_{L^2(\Omega)} = \|u\|_{L^2(\Omega)}^2 + \|\nabla u\|_{L^2(\Omega)}^2.$$

Since $u \in H_0^1(\Omega)$, we can use the Poincaré's inequality:

Lemma 1. *If Ω is bounded, then for constant C ,*

$$\|u\|_{L^2(\Omega)} \leq C(\Omega) \|\nabla u\|_{L^2(\Omega)}, \forall u \in H_0^1(\Omega).$$

Therefore we have:

$$\|u\|_{H^1(\Omega)} = (\|u\|_{L^2(\Omega)}^2 + \|\nabla u\|_{L^2(\Omega)}^2)^{\frac{1}{2}} \leq (1 + C^2)^{\frac{1}{2}} \|\nabla u\|_{L^2(\Omega)}.$$

On the other hand, from:

$$\|u\|_{H^1(\Omega)}^2 = \|u\|_{L^2(\Omega)}^2 + \|\nabla u\|_{L^2(\Omega)}^2,$$

it follows that:

$$\|\nabla u\|_{L^2(\Omega)} \leq \|u\|_{H^1(\Omega)}.$$

As a consequence:

$$\|\nabla u\|_{L^2(\Omega)} \leq \|u\|_{H^1(\Omega)} \leq (1 + C^2)^{\frac{1}{2}} \|\nabla u\|_{L^2(\Omega)}.$$

This shows that the two norms are equivalent on $H_0^1(\Omega)$.

Practical Exercises

Task 5

Given $\Omega := [-\frac{1}{2}, 2]x[-\frac{1}{2}, 1] \subset \mathbb{R}^2$. Set up a Cartesian grid that tessellates Ω with 10 cells in the x direction and 20 in the y direction.

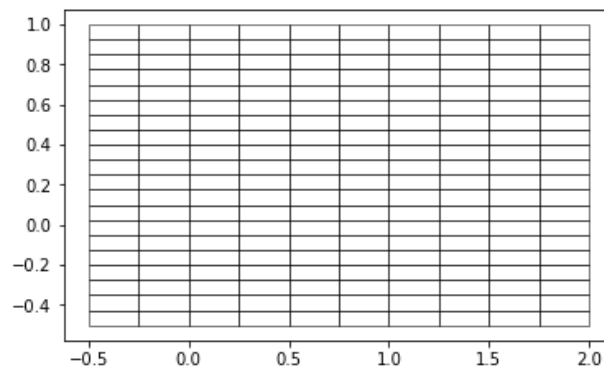
The code we use for this task is as follows:

```
1 from dune.grid import cartesianDomain, yaspGrid
2 domain = cartesianDomain ([-0.5, -0.5], [2,1], [10,20])
3 yaspView=yaspGrid(domain)
4 yaspView.plot()
```

In this problem we setup a Cartesian grid by using `cartesianDomain` and `yaspGrid`.

We, thus, have a rectangular structured domain. After creating this domain, we define it by using the `cartesianDomain` function. Its parameters are the minimum x and y , then the maximum x and y , and the number of cells required. Then, we initialize the structured grid using the domain.

The last line is used to see the plot and we get the following:



Task 6

Tessellate Ω with a triangular grid with twice as many cells as the Cartesian grid.

The code we use for this task is in addition to the code already given under Task 5 and it is as follows :

```
1 from dune.alugrid import aluConformGrid
2 aluView=aluConformGrid(domain)
3 aluView.plot()
```

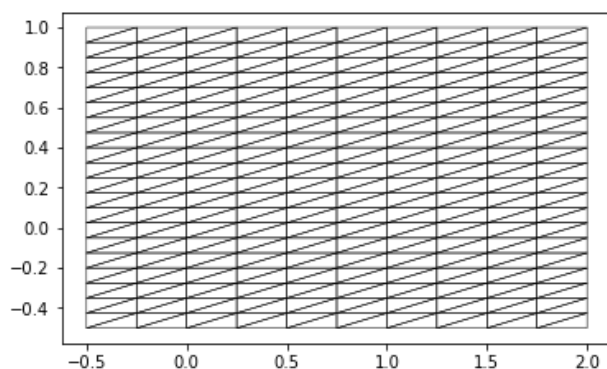
In this problem we need to create a triangular grid.

We used the cartesian grid created in Task 5 and converted it into a triangular grid by splitting each cell into two parts by a diagonal crossing the cell. In this way, two triangles for each cell is generated.

`alugrid` allows us to create an unstructured grid, while `aluConformGrid` creates the triangular grid from the domain we defined in Task 5.

The Cartesian grid in Task 5 had a total of $10 \times 20 = 200$ cells. So, now dividing each cell into two to obtain the triangles, we have a total of $2 \times 200 = 400$ cells. In this way, we create a triangular grid with twice as many cells as the Cartesian grid.

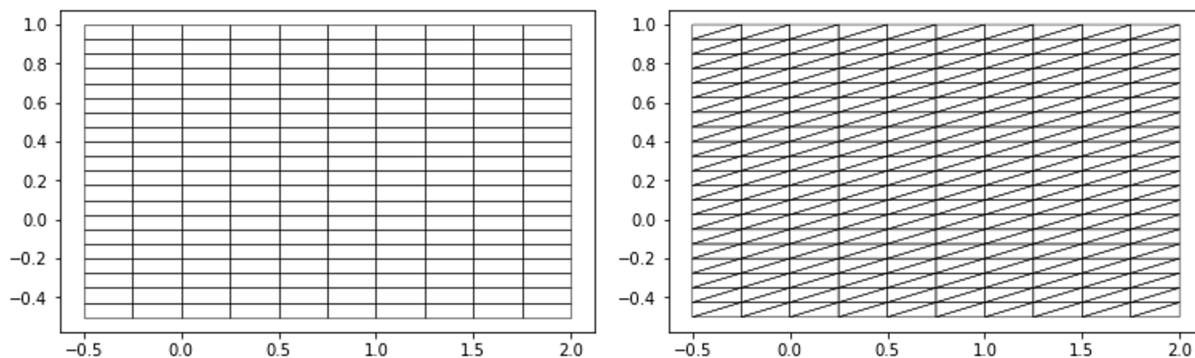
We then get the following plot:



Task 7

Plot both grids.

We have already coded this under Task 5 and 6 and the following are the plots we obtain:



Task 8

Create a triangular grid of the above domain Ω with a hole inside.

In order to do this, we have first used prewritten codes from the given website in the Task question to define `gms2DGF`, which converts a mesh generated by `pygmsh` into `.dgf` format and `projectBoundaryIds`, which uses finite volume spaces or reads the boundary elements to assign boundary IDs. The full code of this task is given in the Appendix A.1.

For this task, we have imported and used `pygmsh`.

```
1 import pygmsh
```

`aluvview` loads a 2D triangular grid from `triangles.dgf` file. Next, we define the same rectangular domain as the previous tasks, $\Omega = [-\frac{1}{2}, 2] \times [-\frac{1}{2}, 1]$ in $\mathbb{R}^2$, and then a hole. The hole is defined as a circle of radius 0.2 with centre at (0.70, 0.20). We use `geom.boolean_difference(rectangle, hole)` to remove the hole from the rectangular domain. We then create the mesh and convert it to `.dgf` format using `gms2DGF`.

```
1 rectangle = geom.add_rectangle([Lx_min, Ly_min, 0], Lx_max - Lx_min,
                                Ly_max - Ly_min)
2 hole = geom.add_ball(hole_center, hole_radius)
3 geom.boolean_difference(rectangle, hole)
4
5 geom.set_mesh_size_callback(size)
6 mesh = geom.generate_mesh(dim=2)
7
8 dgf = gms2DGF(mesh.points, mesh.cells_dict, bndDomain=bndDomain, dim=2)
9 domain2d = (reader.dgfString, dgf)
```

We finally assign boundary IDs using `projectBoundaryIds` and plot the results. We have also plotted the result without boundary IDs. Both plots are given below:

```
1 gridView2d = leafGridView(domain2d, dimgrid=2)
2 bndIds = projectBoundaryIds(gridView2d)
3 bndIds.plot()
4 gridView2d.plot(figsize=(5,5))
```

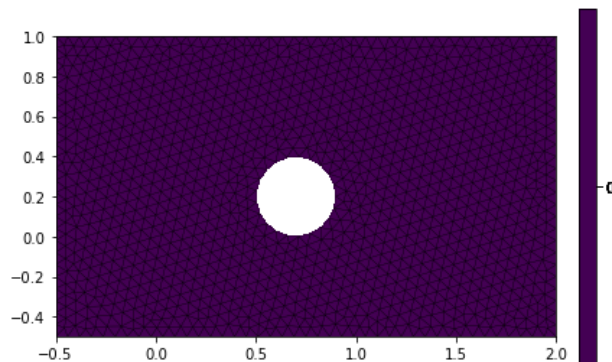


Figure 1: With Boundary Ids assigned

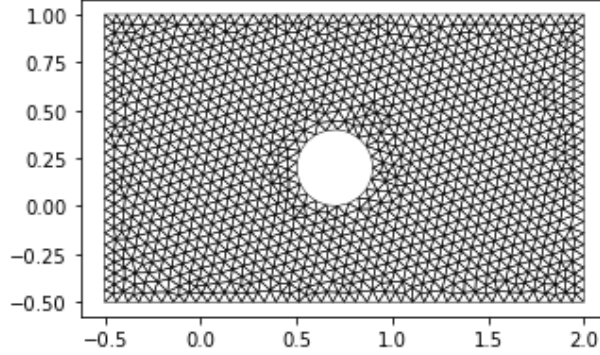


Figure 2: Without Boundary Ids assigned

Task 9

Compute the grid width h of the created grid with:

$$h := \max_{E \in \mathcal{T}_h} h_E. \quad (1)$$

h_E can be chosen in different ways. Choose one and justify your choice. Compare your results with:

```
1 from dune.fem.utility import gridWidth
2 h = gridWidth( gridView )
```

We chose a cartesian domain $\Omega = [-\frac{1}{2}, 2] \times [-\frac{1}{2}, 1]$ in $\mathbb{R}^2$ and we divided into 10 cells in the x - direction and 20 cells in the y -direction. We can write in code form as:

```
1 domain = cartesianDomain([ -0.5, -0.5 ], [ 2, 1 ], [ 10, 20 ])
2 gridView = yaspGrid(domain)
```

After that, we choose to compute h_E as the diameter of E , so the maximum distance between any two points of E . We can compute the difference of the maximum and minimum coordinates:

$$L_x = x_{max} - x_{min} = 2 - \left(-\frac{1}{2}\right) = 2.5$$

$$L_y = y_{max} - y_{min} = 1 - \left(-\frac{1}{2}\right) = 1.5.$$

Where L_x gives the total width of the domain, while L_y returns the total height of the domain. Since we have 10 cells in the x -axis, we will have $\Delta x = \frac{L_x}{10} = 0.25$ whereas for y -axis, we have 20 cells, therefore $\Delta y = \frac{L_y}{20} = 0.075$. The diameter, which is the length of the diagonal, is:

$$h_E = \sqrt{(\Delta x)^2 + (\Delta y)^2}.$$

The maximum is the same for all cells. We can translate it in Dune using the following code:

```
1 dx = (2 - (-0.5)) / 10
```

```

2 dy = (1 - (-0.5)) / 20
3 mh = math.sqrt(dx**2 + dy**2)

```

Our manual computation return a value equals to 0.26100766272276377, while if we use:

```

1 from dune.fem.utility import gridWidth
2 h = gridWidth( gridView )

```

we obtain the value: 0.25. Therefore we obtain approximately the same value. The full code of this task is available in appendix A.2.

Task 10

Write a piecewise constant grid function that on each element of E returns h_E .

We define the same grid as in task 9. After that we write a piecewise constant grid function in the following way:

```

1 @gridFunction(grid, order=0, name="h_E")
2 def h_E(element, local_coordinate):

```

We assign order 0 since the function is a piecewise constant function. We then use the Euclidean distance:

```

1 for i in range(n_corners):
2     for j in range(i + 1, n_corners):
3         p = np.array(corners[i])
4         q = np.array(corners[j])
5         dist = np.linalg.norm(p - q)
6         if dist > max_dist:
7             max_dist = dist
8 return max_dist

```

where we defined corners as:

```

1 geo = element.geometry
2 corners = geo.corners

```

For every element h_E , the returned value is: 0.26100766272276377. The full code of this task is available in appendix A.3.

Task 11

*Create a degrees of freedom (dof) mapper with **vertex layout**. Print the number of dofs and compare with the number of vertices of the grid.*

We first create our domain and the grid using the structured cartesian grid. Then we define `mapper` which maps the degrees of freedom (dof) to grid entities, which in this task are vertices. Then, we find the total number of dofs as 25 which is equal to the number of vertices

as confirmed by finding `numVertices`.

We finally print the results, mainly the dof index, the vertices and its corresponding dof and a list of dof indices.

The full code for this task is as follows:

```

1 dim = 2
2
3 from dune.grid import yaspGrid as hierarchicGrid
4 from dune.grid import cartesianDomain
5 domain = cartesianDomain([0]*dim, [1]*dim, [4]*dim)
6
7 view = hierarchicGrid(domain)
8 mapper = view.mapper(lambda gt: 1 if gt.dim == 0 else 0)
9 nDofs = mapper.size
10 print(f"Total number of DOFs: {nDofs}")
11
12 numVertices = len(view.vertices)
13 print(f"Number of vertices: {numVertices}")
14
15 for element in view.elements:
16     elemIdx = mapper.index(element)
17
18     print(f"\nElement {elemIdx}:")
19     for vertex in element.vertices:
20         vxIdx = mapper.index(vertex)
21         print(f"Vertex {vertex.geometry.center} -> DOF {vxIdx}")
22
23     idx = mapper(element)
24     print(f"Element DOF indices: {idx}")

```

Task 12

Write a function `linearInterpolation` that takes a grid function `u` and a `gridView` and returns the `mapper` and vector of degrees of freedom (`dofs`), i.e., the `numpy` array storing the values of `u` at the vertices. Note: note that a grid function has an attribute `gridView`. Then write a grid function `uh` that returns the linear interpolation for the given dof vector and `mapper`. Use the unstructured triangular grid from before and refine it 3 times globally.

The full code of this task is in Appendix A.4

We start by creating the same 2D domain as the previous tasks, $\Omega = [-\frac{1}{2}, 2] \times [-\frac{1}{2}, 1]$ in $\mathbb{R}^2$, along with a structured cartesian grid using `yaspGrid` and an unstructured triangular grid using `aluConformGrid`. We plot the initial unstructured grid.

```

1 domain = cartesianDomain([-0.5, -0.5], [2,1], [10,20])
2 yaspView = yaspGrid(domain)
3 aluView = aluConformGrid(domain)
4 aluView.plot()
5
6 view = aluview

```

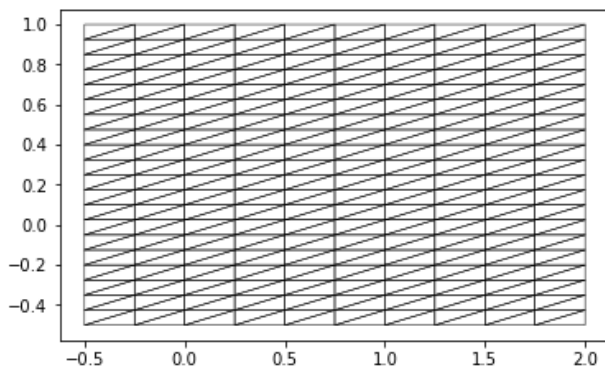


Figure 3: Initial unstructured grid

The grid is then refined globally 3 times. This makes the triangular elements smaller. This refined grid is then plotted.

```

1 aluView.hierarchicalGrid.globalRefine(3)
2 aluView.plot()

```

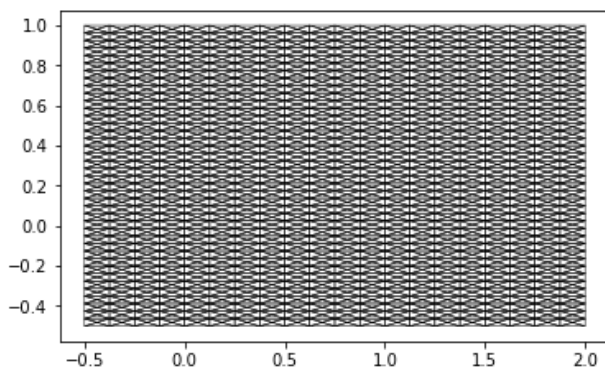


Figure 4: Refined grid

mapper is the same as Task 11. We then define the function **linearInterpolation** as described in the Task question. It takes a grid function **u** and a **gridView** as parameters. In the function, a **numpy array** is initialized then a loop iterates over all vertices and stores the function value of **u** at the vertices into array. The function returns the **mapper** and vector of degrees of freedom (dofs).

```

1 def linearInterpolation(u, gridView):
2
3 mapper = gridView.mapper(lambda gt: 1 if gt.dim == 0 else 0)
4 dofs = np.zeros(mapper.size)
5
6 for vertex in gridView.vertices:
7     dofs[mapper.index(vertex)] = u(vertex.geometry.center)
8
9 return mapper, dofs

```

Our test case: We have defined the function $u(x)$ as $x[0] + x[1]$, a simple linear function. We then plot u .

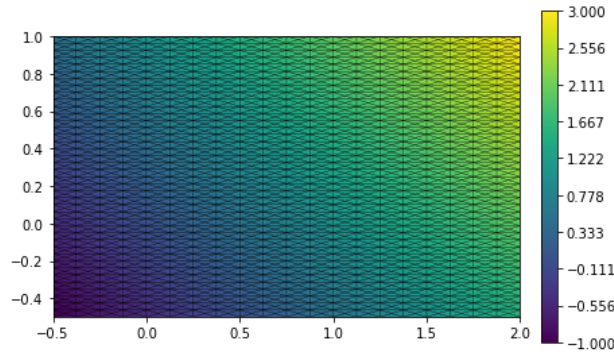


Figure 5: Plot of u

We then compute the interpolation,

```

1 mapper, dofs = linearInterpolation(u, view)

```

and then define the grid function u_h . In this function, we retrieve the vertices of the element and then get the `dofs` for vertices p_0, p_1 and p_2 and assign them into `u_p1`, `u_p0` and `u_p2` respectively. The function returns the interpolation equation: $u_h(e, \hat{x}) = u(p_0)(1 - \hat{x}_0 - \hat{x}_1) + u(p_1)\hat{x}_0 + u(p_2)\hat{x}_1$, where, $\hat{x}_0, \hat{x}_1$ are the coordinates in the reference element $\hat{E}$. We then plot u_h .

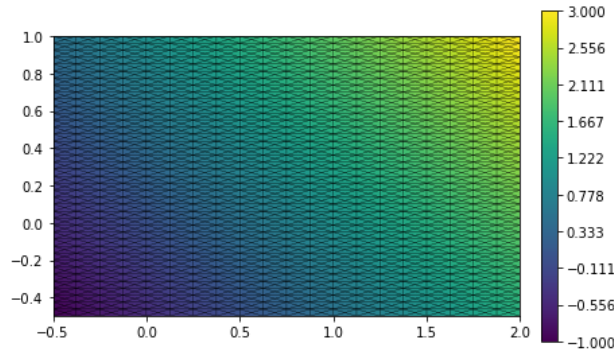


Figure 6: Plot of u_h

Task 13

Compute the maximum interpolation error by comparing the linear interpolation u_h at the center of each element with the value of u . Write a grid function that returns the error for each element E and print the maximal value. Compute

$$e = \max_{E \in \mathcal{T}_h} |u(\omega_E) - u_h(\omega_E)| \quad (2)$$

where ω_E is the center of an element E .

We will solve this task by appending the code below to the code of Task 12. This can be viewed in the Appendix A.4.

In the code below, we have defined grid function `error`. In this, we compute the average (arithmetic mean) of the DOF values at the three vertices and assign it to `uh_center` which is the interpolated approximation. We then evaluate the true function value `u_center` at the geometric center of the element. The function returns the absolute difference between `u_center` and `uh_center`. This function thus measures the error or basically how much the linear interpolation deviates from the true function. We then plot `error` which visualizes the error over the entire grid.

```

1 @gridFunction(view, order=3, name="error")
2 def error(element, ref_coords):
3     # for linear interpolation in a triangle we compute the arithmetic
4       mean of the dofs at the vertices:
5
6     vertex_values = [dofs[mapper.index(vertex)] for vertex in element
7                       .vertices]
8     uh_center = sum(vertex_values) / len(vertex_values)
9     u_center = u(element.geometry.center)
10    return abs(u_center - uh_center)
11
12 error.plot()
```

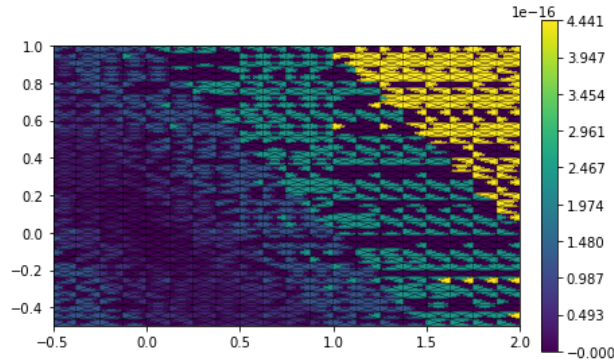


Figure 7: Plot of error

Finally, we compute the maximum interpolation error over all elements which we get as $4.440892098500626e - 16$. This shows that the error is very small, close to being absolutely precise. Or, the linear interpolation of u is almost exactly the same as u_h . Hence, this is also why it can be noticed in Task 12 that the plots of u and u_h are identical to each other.

```

1 max_error = max(
2     abs(u(element.geometry.center) -
3         sum(dofs[mapper.index(vertex)] for vertex in element.vertices) /
4         len(element.vertices))
5     for element in view.elements
6 )
7 print("Initial maximum interpolation error:", max_error)

```

Task 14

Compare the error on two different refinement levels. Is there a relation to h ?

In order to complete this task we compute two times a global refinement, in addition to the three of the previous tasks, using the following two times:

```

1 aluView.hierarchicalGrid.globalRefine(1)
2 aluView.plot()

```

Instead of the previous domain we change it to the following:

```

1 domain = cartesianDomain([0, 0], [1, 1], [1, 1])
2 yaspView = yaspGrid(domain)
3 aluView = aluConformGrid(domain)

```

We decided to use a non-linear function, $x^2 + y^2$ in order to investigate the behavior of the interpolation error. The following table shows the maximum interpolation error after the three initial global refinements (Task 12 and 13) and after two additional global refinements,

which we label "1st Refinement" and "2nd Refinement". We work on a triangular grid whose

Max interpolation error	1st Refinement	2nd Refinement
0.05555555555555569	0.0277777777777778123	0.013888888888889062

initial cell diameter is $h = 0.5$. Thus, we have $h^2 = (0.5)^2 = 0.25$. We want to show that:

$$error \approx C \cdot h^2,$$

where C is a constant. Using the initial error of $0.05555555555555569 \approx 0.05556$, we compute:

$$C = \frac{0.05556}{0.25} = \frac{\text{initial error}}{h^2} \approx 0.22224.$$

Since we have a triangular grid, the cell diameter is reduced by a factor of approximately 0.7071 at each step. Hence, after the first refinement the new cell diameter, h_1 , is:

$$h_1 = h \cdot 0.7071 = 0.3536.$$

It follows that $h_1^2 \approx 0.125$ and the error becomes $error_1 \approx C \cdot h_1^2 \approx 0.02778$. After the second refinement, the cell diameter reduces further to $h_2 = h_1 \cdot 0.7071 \approx 0.25$, therefore $h_2^2 \approx 0.0625$. The error becomes $error_2 \approx C \cdot h_2^2 \approx 0.01389$. These computed values match the ones of our table. The error ratio between each error is about of 0.5, for instance:

$$\frac{0.02778}{0.05556} \approx 0.5,$$

which is consistent with the fact that the cell diameter is reduced each time by 0.7071 (since $(0.7071)^2 \approx 0.5$). Consequently we can conclude that the interpolation error is of order $O(h^2)$. The full code is available in the Appendix A.4.

A Appendix

A.1 Task 8 Full Code:

```
1 import numpy as np
2 import pygmsh
3 from matplotlib import pyplot
4 from dune.alugrid import aluSimplexGrid as leafGridView
5 from dune.grid import OutputType, reader
6 from dune.fem.space import lagrange as solutionSpace
7 from dune.fem.scheme import galerkin as solutionScheme
8 from ufl import TestFunction, TrialFunction, SpatialCoordinate
9 from ufl import dx, grad, grad, dot, inner, conditional, sin
10
11 domain2d = (reader.dgf, "triangles.dgf")
12
13 # specify dimgrid since it cannot be easily extracted from the dgf file
14 aluView = leafGridView(domain2d, dimgrid=2)
15 aluView.plot(figsize=(5,5))
16
17 def gmsh2DGF(points, cells, bndDomain = None, bndSegments=None, periodic =
18             None, dim = None, computeFirstIndex
19             = True):
20
21     """
22     Parameter:
23         points      array(list) of points of length dim
24         cells       dict containing element vertex numbers
25         bndDomain   dict id -> list[lower,upper] (or id -> str) where lower
26                     and upper describe the bounding box
27                     of the boundary section
28         bndSegments dict id -> list of lists containing vertex numbers of
29                     boundary segments
30         periodic    string containing periodic boundary transformation
31         dim         dimension of grid
32         computeFirstIndex if true, first vertex index is computed bases on
33                     all cells vertices (default is on)
34
35     Returns
36         String containing the mesh description in DGF format.
37     """
38     import numpy as np
39
40     if dim is None:
41         if "tetra" in cells or "hexa" in cells:
42             dim = 3
43         elif "quad" in cells or "triangle" in cells:
44             dim = 2
45         else:
46             dim = len(points[0])
47
48     simplex = "triangle" if "triangle" in cells else None
49     if dim == 3 and "tetra" in cells:
```

```

43     simplex = "tetra"
44
45     # default numbering is 0 based
46     firstVertexIndex = 0
47     for key, cellVertices in cells.items():
48         firstVertexIndex = min(firstVertexIndex, np.min(cellVertices))
49
50     dgf="DGF\nVertex\n"
51     # if first vertex index is not 0 we need to add this here
52     if firstVertexIndex > 0:
53         dgf += f"firstindex {firstVertexIndex}\n"
54
55     for p in points:
56         for i in range(dim):
57             dgf += str(p[i]) + " "
58         dgf += "\n"
59     dgf += "#\n\n"
60
61     if simplex is not None:
62         dgf += "Simplex\n"
63         for t in cells[simplex]:
64             for v in t:
65                 dgf += str(v) + " "
66             dgf += "\n"
67         dgf += "#\n\n"
68
69     if "quad" in cells:
70         dgf += "Cube\n"
71         # gmsh has a different reference quadrilateral
72         vxmap = [0,1,3,2] # flip vertex 2 and 3
73         for t in cells["quad"]:
74             for i in range(4):
75                 dgf += str(t[vxmap[i]]) + " "
76             dgf += "\n"
77         dgf += "#\n\n"
78
79     # boundary segments
80     if bndSegments is not None:
81         assert isinstance(bndSegments, dict), "Expecting a dictionary for
82                                     boundary domain"
83         dgf += "BoundarySegments\n"
84         for bndid, bndsegs in bndSegments.items():
85             for segment in bndsegs:
86                 dgf += str(bndid)
87                 for vx in segment:
88                     dgf += " " + str(vx)
89                 dgf += "\n"
90         dgf += "#\n\n"
91
92     # boundary domain section
93     if bndDomain is not None:

```



```

93         assert isinstance(bndDomain, dict), "Expecting a dictionary for
94             boundary domain"
95     dgf += "BoundaryDomain\n"
96     for bndid, bnd in bndDomain.items():
97         if isinstance(bnd, str):
98             if bnd == 'default':
99                 dgf += bnd + " " + str(bndid) + "\n"
100             else:
101                 dgf += str(bndid) + " " + bnd + "\n"
102         else: # tuple or list
103             # has to be either list or tuple here
104             assert isinstance(bnd, (tuple, list))
105             assert len(bnd) == 2 # a lower left and upper right corner
106             dgf += str(bndid)
107             for coord in bnd:
108                 assert len(coord) == dim # should a coordinate in the
109                     domain
110                 for c in coord:
111                     dgf += " " + str(c)
112             dgf += "\n"
113
114     dgf += "#\n"
115
116     # periodic boundaries
117     if periodic is not None:
118         dgf += periodic
119
120     return dgf
121
122 # a simple projection for boundary ids
123 def projectBoundaryIds( gridView ):
124     """
125     Parameter:
126         gridView      a grid view to project boundary ids for
127
128     Returns:
129         a piecewise discrete function containing values corresponding to
130         adjacent boundaries. `0` refers to interior elements.
131     """
132     import io, sys
133     from dune.fem.space import finiteVolume
134     from dune.generator import algorithm
135
136     code = """
137     #include <dune/fem/misc/boundaryidprovider.hh>
138
139     template <class GridView, class Intersection>
140     int boundaryId( const GridView& gv, const Intersection& i )
141     {
142         return Dune::Fem::boundaryId( gv, i );
143     }
144     """

```

```

143
144 space = finiteVolume( gridView )
145 bndFunc = space.interpolate([0], name="bndId" )
146
147 bndId = None
148
149 for e in gridView.elements:
150     idx = gridView.indexSet.index( e )
151     if e.hasBoundaryIntersections:
152         for i in gridView.intersections( e ):
153             if bndId is None:
154                 bndId = algorithm.load("boundaryId", io.StringIO(code)
155                                     , gridView, i )
156             if i.boundary:
157                 # hack: assumption is that index and dof correspond
158                 # which is only the case in this specific situation
159                 bndFunc.dofVector[ idx ] = bndId( gridView, i )
160
161 return bndFunc
162
163 try:
164     with pygmsh.occ.Geometry() as geom:
165
166         # Define domain
167         Lx_min, Lx_max = -0.5, 2
168         Ly_min, Ly_max = -0.5, 1
169         #L = 1; H = 1
170         #circles
171         #c = 0.2; r = 0.0
172         hole_center = [0.70, 0.20, 0]
173         hole_radius = 0.2
174
175         # Grid size function
176         def size(dim, tag, x, y, z, lc):
177             resolution = 0.06
178             d=np.sqrt((x-hole_center[0])**2 + (y-hole_center[1])**2)
179             return resolution
180
181
182         # create rectangle and circles
183         rectangle = geom.add_rectangle([Lx_min, Ly_min, 0], Lx_max -
184                                     Lx_min, Ly_max - Ly_min)
185         hole = geom.add_ball(hole_center, hole_radius)
186
187         # subtract the two obstacles from the volume to create holes
188         geom.boolean_difference(rectangle, hole)
189         geom.set_mesh_size_callback(size)
190         mesh = geom.generate_mesh(dim=2)
191
192         #eps = 0.01 # tolerance
193         # lower left circle min and max

```

```

193     #c1 = [c - r - eps, c + r + eps]
194     # upper right circle min and max
195     #c2 = [(1-c) - r - eps, (1-c) + r + eps]
196     L = Lx_min
197     H = Ly_max
198     # dictionary containing id and a list containing the lower left
                                and upper right corner of the
                                bounding box
199     bndDomain = {#1: [[-1.0 , -1.0 ], [0.0  , H+0.1]], # bounding box
                                inflow
200                  # 2: [[ L , -1.0 ], [L+1.0, H+0.1]], # bounding box
                                outflow
201                  # 4: [[c1[0], c1[0]], [c1[1], c1[1]]], # bounding box
                                lower left circle
202                  # 5: [[c2[0], c2[0]], [c2[1], c2[1]]], # bounding box
                                upper right circle
203                  # 3: "default" # top and bottom wall,
204                  # which are all other segments not contained in the
                                above bounding boxes
205                  }
206
207     # return dgf string which can be read by DGF parser or written to
                                file for later use
208     dgf = gmsh2DGF(mesh.points, mesh.cells_dict, bndDomain=bndDomain,
                                dim=2)
209     domain2d = (reader.dgfString, dgf)
210 except ImportError: # pygmsh not installed - use a simple cartesian domain
211     print("pygmsh module not found using a simple Cartesian domain -
                                ignored")
212     from dune.grid import cartesianDomain
213     domain2d = cartesianDomain([-0.25,-0.25],[0.25,0.25],[30,30])
214
215     gridView2d = leafGridView(domain2d, dimgrid=2)
216
217     bndIds = projectBoundaryIds( gridView2d )
218     bndIds.plot()
219     gridView2d.plot(figsize=(5,5))

```

A.2 Task 9

```

1 import math
2 from dune.grid import cartesianDomain, yaspGrid
3 from dune.fem.utility import gridWidth
4
5
6 domain = cartesianDomain([-0.5, -0.5], [2, 1], [10, 20])
7 #yaspView = yaspGrid(domain)
8 gridView = yaspGrid(domain)
9
10 dx = (2 - (-0.5)) / 10 # = 2.5 / 10 = 0.25
11

```

```

12 dy = (1 - (-0.5)) / 20 # = 1.5 / 20 = 0.075
13
14
15 mh = math.sqrt(dx**2 + dy**2)
16 #manual_h = max(dx, dy) # 0.25
17 print("Manual computations (cell diameter):", mh)
18
19
20 uh = gridWidth(gridView)
21 print("According to Dune is instead:", uh)

```

A.3 Task 10

```

1 import math
2 import numpy as np
3 from dune.grid import cartesianDomain, yaspGrid
4 from dune.fem.function import gridFunction
5
6 # Define the grid
7 domain = cartesianDomain([-0.5, -0.5], [2, 1], [10, 20])
8 grid = yaspGrid(domain)
9
10 @gridFunction(grid, order=0, name="h_E")
11 def h_E(element, local_coordinate):
12
13
14     geo = element.geometry
15     corners = geo.corners
16
17
18     max_dist = 0.0
19     n_corners = len(corners)
20
21     for i in range(n_corners):
22         for j in range(i + 1, n_corners):
23             p = np.array(corners[i])
24             q = np.array(corners[j])
25             dist = np.linalg.norm(p - q)
26             if dist > max_dist:
27                 max_dist = dist
28     return max_dist
29
30
31 for i, element in enumerate(grid.elements):
32     local_coordinate = [0.0, 0.0]
33     value = h_E(element, local_coordinate)
34     print(f"Element {i}: h_E = {value}")

```

A.4 Task 12, 13, 14 Full Code:

```

1 import numpy as np
2 from numpy import sin
3 from dune.fem.function import gridFunction
4 from dune.grid import cartesianDomain, yaspGrid
5 from dune.alugrid import aluConformGrid
6
7
8 #domain = cartesianDomain([-0.5, -0.5], [2, 1], [10, 20])
9 domain = cartesianDomain([0, 0], [1, 1], [1, 1])
10
11 #from dune.alugrid import aluConformGrid as leafGridView
12
13 #mesh = {
14 #     "vertices": [
15 #         [0, 0],
16 #         [1, 0],
17 #         [1, 1],
18 #         [0, 1]
19 #     ],
20 #     "simplices": [
21 #         [0, 1, 2],
22 #         [0, 2, 3]
23 #     ]
24 # }
25
26
27 #view = leafGridView(constructor=mesh, dimgrid=2)
28
29
30
31 yaspView = yaspGrid(domain)
32 aluView = aluConformGrid(domain)
33 #aluView = aluConformGrid(mesh)
34 #aluView.plot()
35
36
37 aluView.hierarchicalGrid.globalRefine(3)
38 #aluView.plot()
39
40
41 view = aluView
42
43
44
45 def cell_diameter(element):
46
47     vertices = [np.array(vertex.geometry.center) for vertex in element.
48                 vertices]
49
50     max_dist = 0.0
51     for i in range(len(vertices)):
52         for j in range(i+1, len(vertices)):
53             dist = np.linalg.norm(vertices[i] - vertices[j])

```

```

52         if dist > max_dist:
53             max_dist = dist
54     return max_dist
55
56
57 diameters = [cell_diameter(element) for element in view.elements]
58 h_initial = max(diameters)
59 print("Minimum cell diameter:", min(diameters))
60 print("Maximum cell diameter:", max(diameters))
61
62
63
64 @gridFunction(view, order=3, name="u")
65 def u(x):
66     # a linear function, change also order = 1
67     #return x[0]+x[1]
68     # a nonlinear function, the order is 3 in this case
69     return x[0]**2 + x[1]**2
70
71
72 u.plot()
73
74
75 def linearInterpolation(u, gridView):
76
77     mapper = gridView.mapper(lambda gt: 1 if gt.dim == 0 else 0)
78     dofs = np.zeros(mapper.size)
79     for vertex in gridView.vertices:
80         dofs[mapper.index(vertex)] = u(vertex.geometry.center)
81     return mapper, dofs
82
83
84 mapper, dofs = linearInterpolation(u, view)
85
86
87
88 @gridFunction(view, order=1, name="uh")
89 def uh(element, ref_coords):
90
91     u_p0 = dofs[mapper.index(element.vertices[0])]
92     u_p1 = dofs[mapper.index(element.vertices[1])]
93     u_p2 = dofs[mapper.index(element.vertices[2])]
94     x_hat0, x_hat1 = ref_coords
95     return u_p0 * (1 - x_hat0 - x_hat1) + u_p1 * x_hat0 + u_p2 * x_hat1
96
97 uh.plot()
98
99
100 @gridFunction(view, order=3, name="error")
101 def error(element, ref_coords):
102

```

```

103         vertex_values = [dofs[mapper.index(vertex)] for vertex in element
104                             .vertices]
105         uh_center = sum(vertex_values) / len(vertex_values)
106         u_center = u(element.geometry.center)
107         return abs(u_center - uh_center)
108 #error.plot()
109
110
111 max_error = max(
112     abs(u(element.geometry.center) -
113         sum(dofs[mapper.index(vertex)] for vertex in element.vertices) /
114           len(element.vertices))
115     for element in view.elements
116 )
117 print("Initial maximum interpolation error:", max_error)
118
119
120 # one additional global refinement (first time)
121 aluView.hierarchicalGrid.globalRefine(1)
122 aluView.plot()
123
124 view = aluView
125
126 # after refinement, the grid (view) has changed.
127 # we recompute the DOFs on the new refined grid:
128 mapper, dofs = linearInterpolation(u, aluView)
129
130 # we redefine the grid functions so we have the new mapper and dofs
131
132 @gridFunction(view, order=1, name="uh_refined")
133 def uh_refined(element, ref_coords):
134     u_p0 = dofs[mapper.index(element.vertices[0])]
135     u_p1 = dofs[mapper.index(element.vertices[1])]
136     u_p2 = dofs[mapper.index(element.vertices[2])]
137     x_hat0, x_hat1 = ref_coords
138     return u_p0 * (1 - x_hat0 - x_hat1) + u_p1 * x_hat0 + u_p2 * x_hat1
139
140 uh_refined.plot()
141
142 @gridFunction(view, order=3, name="error_refined")
143 def error_refined(element, ref_coords):
144     vertex_values = [dofs[mapper.index(vertex)] for vertex in element.
145                       vertices]
146     uh_center = sum(vertex_values) / len(vertex_values)
147     u_center = u(element.geometry.center)
148     return abs(u_center - uh_center)
149 error_refined.plot()
150
151

```

```

152 max_error_refined = max(
153     abs(u(element.geometry.center) -
154         sum(dofs[mapper.index(vertex)] for vertex in element.vertices) /
155         len(element.vertices))
156     for element in view.elements
157 )
158 print("After first global refinement, maximum interpolation error:",
159       max_error_refined)
160
161 diameters_refined = [cell_diameter(element) for element in view.elements]
162 h_refined = max(diameters_refined)
163 print("After the first refinement, minimum cell diameter:", min(
164     diameters_refined))
165 print("After the first refinement, maximum cell diameter (h):", h_refined)
166
167 error_ratio = max_error_refined / max_error
168 h_ratio = h_refined / h_initial
169 expected_error_ratio = h_ratio**2
170 print("the error ratio after refinement (max_error_refined / max_error):",
171       error_ratio)
172 print("cell diameter ratio (h_refined / h_initial):", h_ratio)
173 print("expected error ratio (h_ratio squared):", expected_error_ratio)
174
175 # second global refinement (second time)
176 aluView.hierarchicalGrid.globalRefine(1)
177 aluView.plot()
178
179 mapper, dofs = linearInterpolation(u, aluView)
180
181
182
183 @gridFunction(view, order=1, name="uh_refined")
184 def uh_refined(element, ref_coords):
185     u_p0 = dofs[mapper.index(element.vertices[0])]
186     u_p1 = dofs[mapper.index(element.vertices[1])]
187     u_p2 = dofs[mapper.index(element.vertices[2])]
188     x_hat0, x_hat1 = ref_coords
189     return u_p0 * (1 - x_hat0 - x_hat1) + u_p1 * x_hat0 + u_p2 * x_hat1
190
191 uh_refined.plot()
192
193 @gridFunction(view, order=3, name="error_refined")
194 def error_refined(element, ref_coords):
195     vertex_values = [dofs[mapper.index(vertex)] for vertex in element.
196                     vertices]
197     uh_center = sum(vertex_values) / len(vertex_values)
198     u_center = u(element.geometry.center)

```



```

199     return abs(u_center - uh_center)
200
201 error_refined.plot()
202
203
204 max_error_refined = max(
205     abs(u(element.geometry.center) -
206         sum(dofs[mapper.index(vertex)] for vertex in element.vertices) /
207           len(element.vertices))
208     for element in view.elements
209 )
209 print("After second refinement, maximum interpolation error:",
        max_error_refined)

```